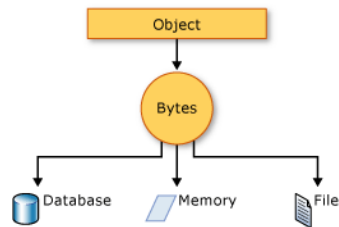


Serialisierung

Was ist Serialisierung?

- Serialisierung ist der Prozess der Konvertierung eines Objekts in eine schreibbare Form
- Wird verwendet um ein Objekt in eine Datenbank, Arbeitsspeicher oder Datei zu übertragen
- Gängige Formate: JSON, XML, CSV, ...



JavaScript Object Notation

- Standardkonforme Funktionen mit geringem Speicherbedarf zur Verarbeitung von Objekten $\leftrightarrow$ JSON
- Serialisierung und Deserialisierung mit integrierter UTF-8 Unterstützung
- Häufig zum Freigeben von Daten über das Internet verwendet wird. (Services)
- Bei der JSON-Serialisierung werden die öffentlichen Properties eines Objekts in eine Zeichenfolge, ein Bytearray oder Stream serialisiert, die der [RFC 8259 JSON-Spezifikation](#) entsprechen

(JavaScript Object Notation)

System.Text.Json

- .NET interne JSON-Implementation
- JsonSerializer.Serialize
- JsonSerializer.Deserialize
- vor .NET Core 3.0 das [System.Text.Json](#) NuGet Paket installieren

```
string json = JsonSerializer.Serialize(fahrzeuge);  
File.WriteAllText(filePath, json);  
  
string readJson = File.ReadAllText(filePath);  
Fahrzeug[] readFzg = JsonSerializer.Deserialize<Fahrzeug[]>(readJson);
```

(JavaScript Object Notation)

JsonSerializerOptions

- Optionen, die bei der Serialisierung/Deserialisierung mitgegeben werden können
- Hier können verschiedene Einstellungen getätigt werden, die die gesamte Json-Datei betrifft
 - Tausendertrennzeichen/Dezimalzeichen
 - Zeilenumbrüche
 - Datumsformate
 - Converter
 - ...

```
JsonSerializerOptions options = new JsonSerializerOptions();  
options.WriteIndented = true;
```

```
string json2 = JsonSerializer.Serialize(fahrzeuge, options);  
File.WriteAllText(filePath, json2);
```

JSON-Attribute

- **JsonIgnore**: Ignoriert das gekennzeichnete Property
- **JsonPropertyName**: Benennt das Property um
- **JsonPropertyOrder**: Ändern die Reihenfolge von Properties
- **JsonExtensionData**: Fängt Felder auf, die in der entsprechenden Klasse kein Property haben
 - Muss auf ein Dictionary angewandt werden
 - Das Dictionary wird erstellt, wenn dieses Attribut verwendet wird
- **JsonDerivedType**: Vererbung ermöglichen
 - Der Typ des Objekts wird bei jedem Json-Objekt dazugeschrieben (\$Type)
 - Benötigt einen String als Parameter (TypeDiscriminator)
 - Wird in die Datei geschrieben

JSON per Hand

- Bei unbekannten JSON-Daten kann die JSON Datei per Hand iteriert werden
- Vorallem bei Daten aus dem Internet oft notwendig
- Benötigte Klassen: JsonDocument, JsonElement

```
JsonDocument doc = JsonDocument.Parse(json2);  
foreach (JsonElement element in doc.RootElement.EnumerateArray())  
{  
    int maxV = element.GetProperty("Maximalgeschwindigkeit").GetInt32();  
    FahrzeugMarke marke = (FahrzeugMarke) element.GetProperty("Marke").GetInt32();  
    Console.WriteLine($"MaxV: {maxV}, Marke: {marke}");  
}
```

Newtonsoft.Json

- Externe JSON-Implementation
- Deutlich älter als System.Text.Json
 - Weniger performant als System.Text.Json
- NuGet: Newtonsoft.Json
- Hat für System.Text.Json den Aufbau vorgegeben
 - Starke Ähnlichkeiten zw. den beiden Frameworks
 - Code von Framework A kann für Framework B fast wiederverwendet werden



XmlSerializer

- Bei der XML-Serialisierung werden die öffentlichen Felder und Eigenschaften eines Objektes serialisiert
- Benötigt ein Objekt des Typs XmlSerializer
 - Dieses benötigt als Parameter den zu Serialisierenden Typen
- Kann nur direkt mit einem Stream arbeiten (u.a. FileStream)
- SOAP-Spezifikation kann serialisiert werden
 - SOAP ist ein auf XML basierendes Protokoll

```
XmlSerializer xml = new XmlSerializer(fahrzeuge.GetType());  
using (StreamWriter sw = new StreamWriter(filePath))  
{  
    xml.Serialize(sw, fahrzeuge);  
}
```

XML-Attribute

- XmlIgnore: Ignoriert das gekennzeichnete Property
- XmlAttribute: Definiert, dass das gekennzeichnete Property in der Attributsform serialisiert wird
- XmlInclude: Ermöglicht Vererbung (benötigt als Parameter die Unterklassen)

```
[XmlInclude(typeof(Fahrzeug))]  
[XmlInclude(typeof(PKW))]  
35 references | ppedvLukasK, 1 day ago | 1 author, 1 change  
public class Fahrzeug  
{  
    [XmlIgnore]  
    [XmlAttribute]  
    1 reference | ppedvLukasK, 1 day ago | 1 author, 1 change  
    public int MaxV { get; set; }  
}
```

XML per Hand

- Bei unbekannten XML-Daten kann die XML Datei per Hand iteriert werden
- Vorallem bei Daten aus dem Internet oft notwendig
- Benötigte Klassen: XmlDocument, XmlNode

```
XmlDocument doc = new XmlDocument();
doc.Load(filePath);
foreach (XmlNode node in doc.DocumentElement)
{
    int maxV = int.Parse(node["MaxV"].InnerText);
    FahrzeugMarke marke = Enum.Parse<FahrzeugMarke>(node.Attributes["Marke"].InnerText);
    Console.WriteLine($"MaxV: {maxV}, Marke: {marke}");
}
```